

Heitor Gabriel Barroso Reis

**Plataforma C- e Processador ARM-like:  
Relatório Técnico Inicial para o Laboratório de  
Sistemas Operacionais**

São José dos Campos - Brasil

Setembro de 2026

Heitor Gabriel Barroso Reis

**Plataforma C- e Processador ARM-like:  
Relatório Técnico Inicial para o Laboratório de Sistemas  
Operacionais**

Relatório inicial que consolida a plataforma de compilação C- e o processador ARM-like existente, estabelece o estado técnico verificado do projeto e registra as lacunas que antecedem o desenvolvimento do Laboratório de Sistemas Operacionais.

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Setembro de 2026

# Resumo

Este relatório estabelece a linha de base técnica para o próximo Laboratório de Sistemas Operacionais. A análise parte de dois trabalhos anteriores: o relatório do processador ARM-like, concluído em 2024, e o relatório de integração do compilador C-, concluído em 2025. Como o hardware foi modificado depois dessas entregas, as descrições históricas foram confrontadas com o RTL, o projeto Quartus, o compilador, o montador e o simulador presentes no repositório em setembro de 2026. O documento consolida somente informações confirmadas, descreve a interface entre código C-, código de máquina e FPGA e identifica divergências relevantes em memória, inicialização, temporização, desvios e diagramas. O Sistema Operacional ainda não existe; por isso, escalonamento, processos, chamadas de sistema, memória virtual e sistema de arquivos são apresentados apenas como lacunas futuras, sem alegação de implementação.

**Palavras-chaves:** Processador. Verilog. FPGA. Compiladores. C-. Arquitetura ARM-like. Sistemas Operacionais. Linha de base.

# Lista de Figuras

|                                                                                  |    |
|----------------------------------------------------------------------------------|----|
| Figura 1 – Comparação conceitual entre organizações Harvard e von Neumann . .    | 10 |
| Figura 2 – Fluxo integrado de C- até o código binário do processador . . . . .   | 12 |
| Figura 3 – Kit FPGA DE2-115 utilizado pelo projeto . . . . .                     | 13 |
| Figura 4 – Captura arquivada da ULA para diferentes opcodes e flags Z/N . . . .  | 15 |
| Figura 5 – Hierarquia dos módulos do compilador e suas relações atuais . . . . . | 18 |
| Figura 6 – Decomposição funcional do montador e do codificador binário . . . . . | 22 |
| Figura 7 – Correspondência validada entre assembly e código de máquina . . . . . | 23 |

# Lista de Tabelas

|          |                                                               |    |
|----------|---------------------------------------------------------------|----|
| Tabela 1 | – Interface externa confirmada no módulo <b>Processor</b>     | 13 |
| Tabela 2 | – Campos atuais da instrução                                  | 14 |
| Tabela 3 | – Operações aceitas pelo montador e modeladas no processador  | 14 |
| Tabela 4 | – Semântica atual de branch                                   | 16 |
| Tabela 5 | – Módulos da fase de análise                                  | 19 |
| Tabela 6 | – Síntese da comparação entre os relatórios e o projeto atual | 25 |

# Sumário

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>INTRODUÇÃO</b>                                  | <b>7</b>  |
| 1.1      | Objetivo                                           | 7         |
| 1.2      | Método e fontes de verdade                         | 7         |
| <b>2</b> | <b>FUNDAMENTAÇÃO PRESERVADA</b>                    | <b>9</b>  |
| 2.1      | Organização Harvard e orientação RISC              | 9         |
| 2.2      | Compilação como contrato entre software e hardware | 10        |
| <b>3</b> | <b>ESTADO ATUAL DO PROCESSADOR</b>                 | <b>13</b> |
| 3.1      | Projeto FPGA e interface externa                   | 13        |
| 3.2      | Formato da instrução                               | 14        |
| 3.2.1    | Semântica da ULA e flags                           | 14        |
| 3.3      | Caminho de dados verificado                        | 15        |
| 3.4      | Registradores, flags e controle de fluxo           | 15        |
| 3.5      | Memórias e temporização                            | 16        |
| <b>4</b> | <b>ESTADO ATUAL DO COMPILADOR</b>                  | <b>17</b> |
| 4.1      | Arquitetura dos módulos                            | 17        |
| 4.2      | Fase de análise                                    | 19        |
| 4.2.1    | Análise léxica e controle de erros                 | 19        |
| 4.2.2    | Análise sintática e AST                            | 19        |
| 4.2.3    | Símbolos, escopos e análise semântica              | 19        |
| 4.3      | IR, back-end e montagem                            | 20        |
| 4.3.1    | Representação intermediária                        | 20        |
| 4.3.2    | Frames, registradores e chamadas                   | 20        |
| 4.3.3    | Montagem e codificação binária                     | 21        |
| 4.3.4    | Exemplo validado de assembly e código de máquina   | 23        |
| 4.4      | Integração com ROM e modelo executável             | 23        |
| 4.5      | Verificação reproduzida nesta consolidação         | 23        |
| <b>5</b> | <b>COMPARAÇÃO DOS RELATÓRIOS E CORREÇÕES</b>       | <b>25</b> |
| <b>6</b> | <b>LACUNAS PARA O PRÓXIMO LABORATÓRIO</b>          | <b>26</b> |
| 6.1      | Lacunas do hardware e da documentação              | 26        |
| 6.2      | Lacunas de interface necessárias a um SO           | 26        |
| 6.3      | Critérios mínimos para a próxima revisão           | 27        |

---

|          |                                             |           |
|----------|---------------------------------------------|-----------|
| <b>7</b> | <b>CONCLUSÃO</b> . . . . .                  | <b>28</b> |
|          | <b>REFERÊNCIAS BIBLIOGRÁFICAS</b> . . . . . | <b>29</b> |

# 1 Introdução

O projeto reúne duas camadas que precisam concordar: um processador didático descrito em Verilog e uma cadeia de compilação que traduz programas C- para a linguagem de máquina desse processador. O relatório de Arquitetura e Organização de Computadores documentou a primeira versão integrada do hardware, enquanto o relatório de Compiladores passou a documentar também o front-end, a representação intermediária, o back-end e o montador. Ambos preservam decisões importantes, mas nenhum relatório histórico, isoladamente, constitui a especificação da versão atual.

Este documento tem três funções. A primeira é registrar a fundamentação que continua válida: organização Harvard, orientação RISC, instruções fixas de 32 bits, banco de registradores, execução condicional e separação entre front-end e back-end ([PATTERSON; HENNESSY, 2017](#); [PATTERSON; HENNESSY, 2014](#); [LOUDEN; SILVA, 2004](#)). A segunda é descrever a implementação observável no repositório. A terceira é identificar o que falta antes de iniciar um Sistema Operacional, sem antecipar funcionalidades que ainda não foram projetadas ou implementadas.

## 1.1 Objetivo

O objetivo é oferecer uma base única e rastreável para o próximo laboratório. A consolidação não é uma simples união textual: uma afirmação antiga só é mantida quando concorda com as fontes atuais. Quando há conflito, registra-se o comportamento atual e a divergência. Quando não existe evidência suficiente, a lacuna permanece explícita.

## 1.2 Método e fontes de verdade

A auditoria adotou a seguinte precedência:

1. RTL e configuração Quartus em `processor/Processor/`;
2. compilador, montador e simulador em `compiler/` e `tools/`;
3. testes executáveis e artefatos reproduzíveis;
4. documentação corrente;
5. relatórios e imagens anteriores, usados como contexto histórico.

As fontes documentais comparadas foram:



- relatório histórico do processador: <reports/processador/processadorARM\_2024\_2.pdf>;
- complemento técnico de codificação, localizado em <reports/processador/>;
- relatório-base de compiladores e de formatação: <Relatorio\_AOC.tex>, localizado em <reports/compiladores/>.

A matriz detalhada de rastreabilidade acompanha este projeto em COMPARACAO.md.

## 2 Fundamentação preservada

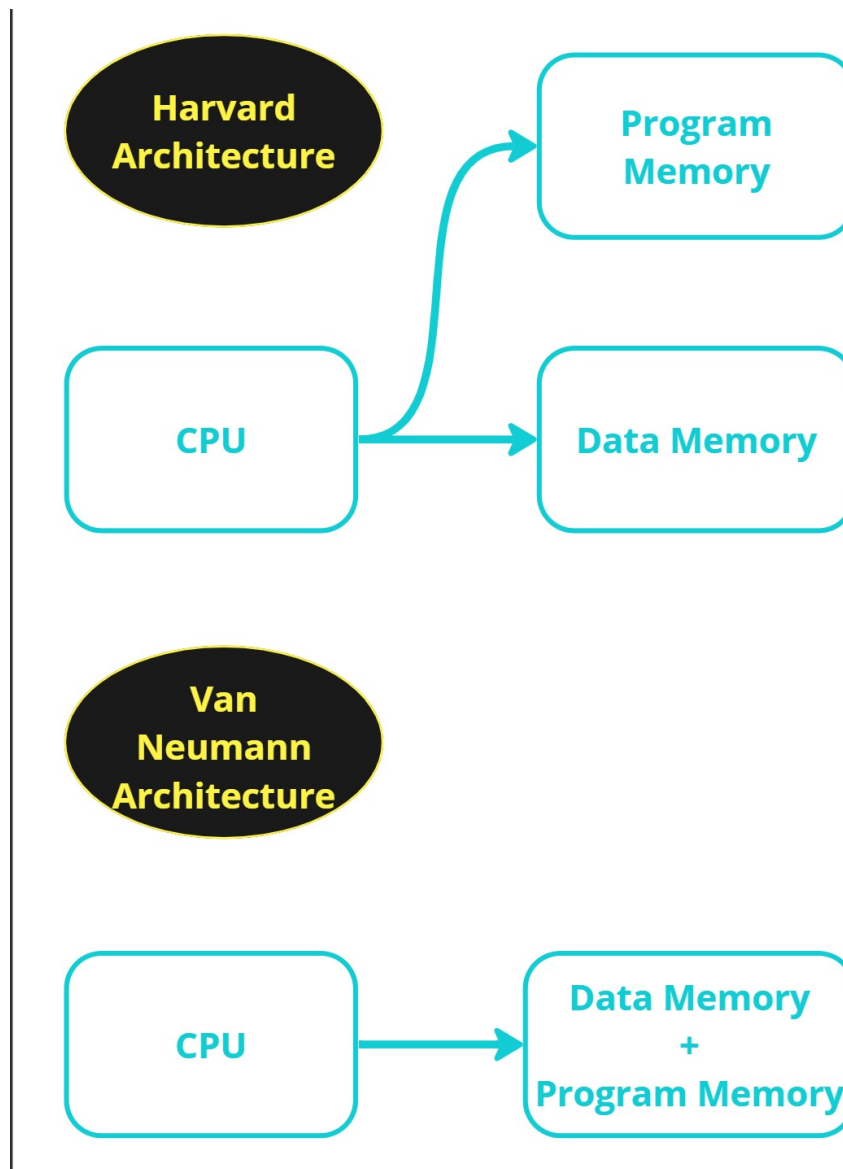
### 2.1 Organização Harvard e orientação RISC

A separação entre memória de instruções e memória de dados caracteriza uma organização Harvard. No projeto atual, a ROM é endereçada pelo contador de programa, enquanto a RAM recebe endereços e dados do caminho de execução. Essa separação permanece coerente com os relatórios anteriores e com o RTL ([PATTERSON; HENNESSY, 2014](#); [HARRIS; HARRIS, 2016](#)).

O processador é ARM-like, e não uma implementação binariamente compatível com ARM. A inspiração aparece na largura fixa das instruções, no campo de condição, no registrador de estado e em operações de branch com link. A codificação, a quantidade de flags e a semântica de memória são próprias do projeto ([ARM, 2001](#)).

A [Figura 1](#) resume a diferença conceitual que fundamenta a organização adotada. Ela não representa as conexões internas do processador; apenas evidencia que, em Harvard, programa e dados ocupam memórias distintas. No projeto, essa separação se concretiza nos módulos `InstructionMemory` e `DataMemory`.

Figura 1 – Comparação conceitual entre organizações Harvard e von Neumann



Fonte: acervo do relatório de compiladores. Figura conceitual validada contra a separação de memórias do RTL atual.

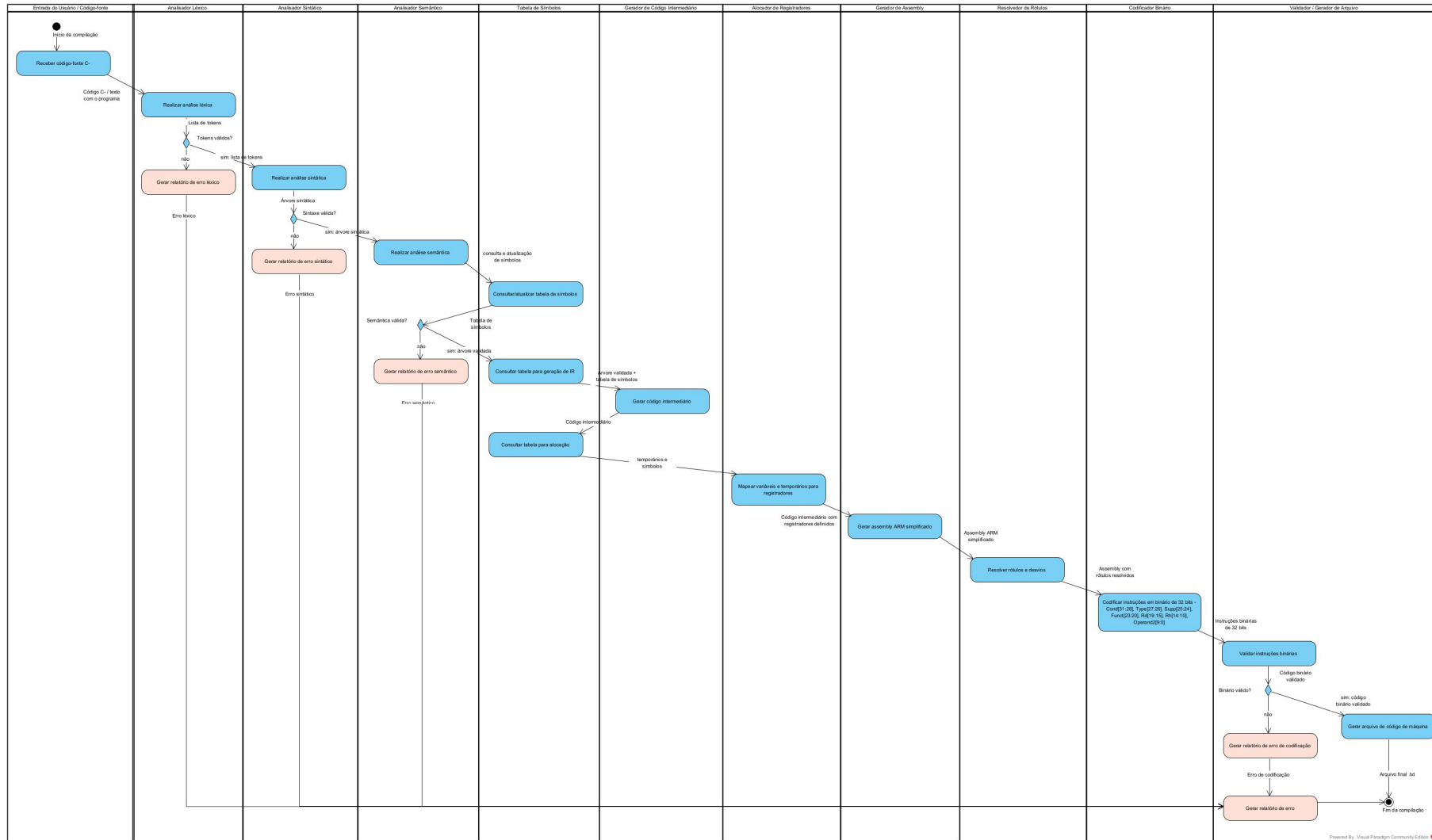
## 2.2 Compilação como contrato entre software e hardware

O compilador transforma C- em uma AST, valida o programa, produz IR textual, gera assembly e codifica palavras de 32 bits. A saída só é correta quando montador, simulador e RTL interpretam os mesmos campos. Portanto, a ISA é uma interface compartilhada e qualquer alteração nela exige validação coordenada.

A [Figura 2](#) organiza esse contrato como uma sequência ponta a ponta. A figura foi preservada do relatório de compiladores porque suas etapas continuam presentes no repositório atual. Os nomes das raias representam responsabilidades lógicas: no front-end,

várias delas cooperam dentro do mesmo executável em C; no back-end, alocação, geração de assembly e parte do gerenciamento de símbolos estão reunidos em módulos Python. Assim, o diagrama descreve o fluxo de informação, e não processos independentes.

Figura 2 – Fluxo integrado de C- até o código binário do processador



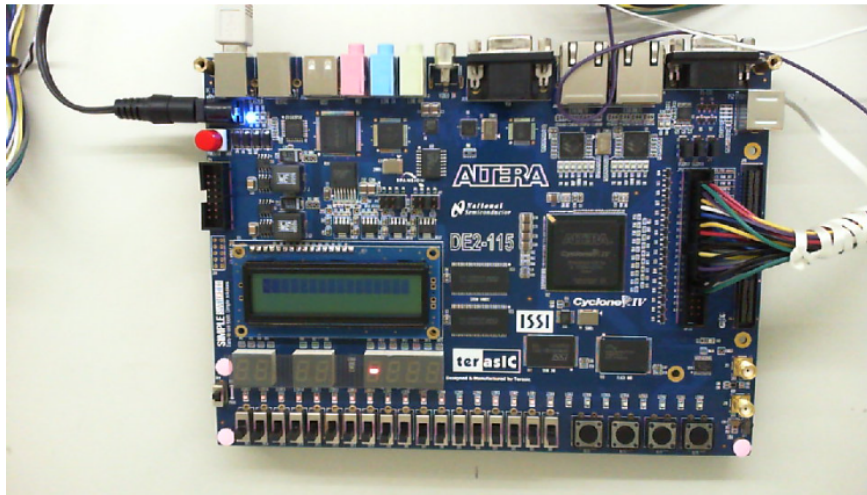
Fonte: relatório de compiladores, validado contra `compiler/main.c`, `compiler/src/`, `compiler/codegen/` e `tools/run_machine_code.py`.

## 3 Estado atual do processador

### 3.1 Projeto FPGA e interface externa

O projeto Quartus seleciona a família Cyclone IV E, o dispositivo EP4CE115F29C7 e a entidade de topo `Processor`. A pinagem corresponde ao kit DE2-115 (TERASIC, 2012). O módulo de topo recebe `CLOCK_50`, chaves e botões; expõe oito displays de sete segmentos, um LED, o último valor de saída e o PC.

Figura 3 – Kit FPGA DE2-115 utilizado pelo projeto



Fonte: acervo do relatório anterior, arquivo `reports/compiladores/Imagens/FPGA.png`.

Tabela 1 – Interface externa confirmada no módulo `Processor`

| Sinal     | Função observada                                                                                        |
|-----------|---------------------------------------------------------------------------------------------------------|
| SW[17]    | Bloqueia o clock efetivo quando está em nível alto.                                                     |
| SW[16]    | Reinicia o PC e o detector associado à entrada; outros bancos não possuem reset explícito no RTL atual. |
| SW[15]    | Confirma a instrução <code>in</code> ; a espera termina em uma transição de descida.                    |
| SW[7:0]   | Valor periférico, estendido para 32 bits na conexão de topo.                                            |
| LEDR[0]   | Resultado da verificação da condição da instrução.                                                      |
| HEX6-HEX7 | Dois dígitos decimais menos significativos do último valor de <code>out</code> .                        |
| HEX0-HEX3 | Representação decimal do endereço corrente de instrução.                                                |

Fonte: O autor, com base em `Processor.v`, `output_module.v` e `Processor.qsf`.

## 3.2 Formato da instrução

O formato confirmado possui 32 bits e é compartilhado pelo montador e pelo decodificador do RTL.

Tabela 2 – Campos atuais da instrução

| Campo    | Bits  | Semântica                                                        |
|----------|-------|------------------------------------------------------------------|
| Cond     | 31:28 | Condição verificada contra Z e N.                                |
| Type     | 27:26 | 00: dados; 01: memória; 11: branch; 10: não utilizado.           |
| Supp     | 25:24 | Bit 25 seleciona imediato; bit 24 solicita atualização do CPSR.  |
| Funct    | 23:20 | Operação; em branches, os bits 23 e 22 controlam link e retorno. |
| Rd       | 19:15 | Registrador geral de destino.                                    |
| Rh       | 14:10 | Primeiro operando ou dado de store.                              |
| Operand2 | 9:0   | Imediato com sinal ou Ro em 9:5, com zeros em 4:0.               |

Fonte: O autor, com base em `ControlUnit.v`, `assembler.py` e `run_machine_code.py`.

Tabela 3 – Operações aceitas pelo montador e modeladas no processador

| Classe   | Operações                                                                    |
|----------|------------------------------------------------------------------------------|
| Dados    | add, sub, mul, div, and, or, xor, not, mov, in, out.                         |
| Memória  | load e store, com endereço fornecido por registrador no fluxo gerado.        |
| Controle | b, bl e o retorno codificado pelo montador como branch para o link dedicado. |

Fonte: O autor.

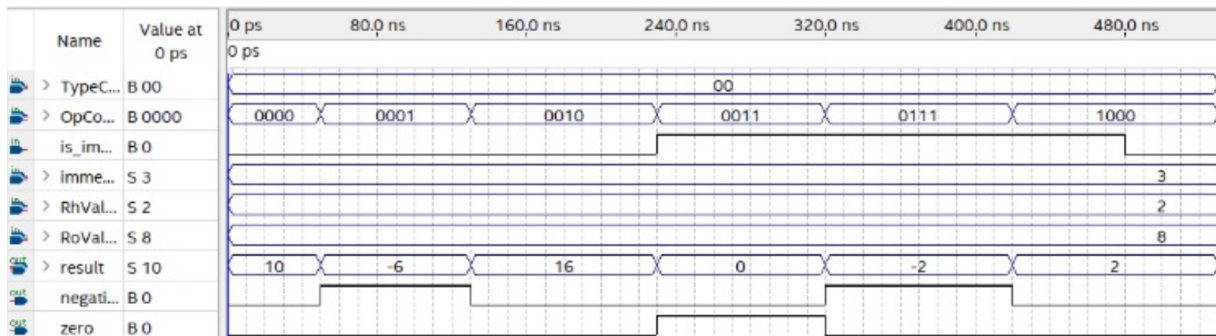
As condições implementadas são `do`, `eq`, `neq`, `gt`, `gteq`, `lt` e `lteq`. Apenas Z e N são produzidas; C e V permanecem zeradas. A condição histórica NOT não é aceita pelo montador e cai no caso falso do `FlagVerifier`; por isso, não integra a ISA atual.

### 3.2.1 Semântica da ULA e flags

A ULA recebe `RhValue` como primeiro operando e seleciona entre `RoValue` e o imediato estendido para o segundo. Para operações de dados, os opcodes 0000–0110 implementam adição, subtração, multiplicação, divisão, AND, OR e XOR. O opcode 0111, embora exposto pelo montador com o mnemônico `not`, executa `-A` em `alu.v`; portanto, sua semântica atual é negação aritmética, não complemento bit a bit. MOV, IN e OUT ocupam 1000, 1001 e 1010.

A Figura 4 é uma captura arquivada cuja sequência foi conferida contra essa lógica combinacional. Ela mostra, para `RhValue=2` e `RoValue=8`, resultados 10, -6, 16 e 0 para ADD, SUB, MUL e DIV, seguidos por -2 no opcode 0111 e 2 em MOV. A figura é evidência ilustrativa compatível, mas não substitui um testbench autochecking atual nem demonstra fechamento de timing.

Figura 4 – Captura arquivada da ULA para diferentes opcodes e flags Z/N



Fonte: acervo do relatório de compiladores, conferido contra `ALUControl.v` e `alu.v`.

### 3.3 Caminho de dados verificado

O caminho de dados ativo foi reconstruído a partir de `integrated.v`. A instrução sai da ROM e é particionada pela unidade de controle. O banco fornece `RhValue`, `RoValue` e o link dedicado. O controlador da ULA escolhe entre `RoValue` e o imediato estendido; a ULA executa a operação ou encaminha o operando necessário a memória e branch. O multiplexador de escrita escolhe o resultado da ULA ou a saída da RAM. A condição, combinada com o tipo da instrução, habilita escritas no banco, no link, na RAM e a atualização do PC.

Não há, no acervo atual, uma figura validada desse caminho. O diagrama histórico foi rejeitado porque representa incremento de PC por quatro, endereço de RAM oriundo da ULA e campos de instrução incompatíveis com o RTL atual. Até que seja produzido um novo diagrama diretamente da implementação, esta descrição textual e as conexões em `integrated.v` são a referência.

### 3.4 Registradores, flags e controle de fluxo

O banco contém 32 registradores gerais de 32 bits e um registrador de link dedicado. Escritas ocorrem na borda positiva do clock efetivo; leituras são registradas na borda positiva de `fast_clock`. O sinal de reset existe na interface do banco, mas não é utilizado internamente, de modo que não se deve afirmar que todos os registradores são zerados no reset.

O PC é atualizado na borda negativa do clock. Se a condição falha, ele avança uma palavra. Se a condição passa, um branch imediato soma ao PC um deslocamento com sinal; um branch por registrador usa diretamente `RoValue`; e a forma de retorno usa o link dedicado. `b1` salva `PC+1` na borda positiva antes da atualização do PC.



Tabela 4 – Semântica atual de branch

| Forma            | Efeito quando a condição é verdadeira                                   |
|------------------|-------------------------------------------------------------------------|
| Imediata         | $PC := PC + \text{signExtend}(\text{imm10}).$                           |
| Por registrador  | $PC := \text{RoValue}.$                                                 |
| Branch-and-link  | $\text{Link} := PC+1$ , seguido do destino imediato ou por registrador. |
| Retorno dedicado | $PC := \text{LinkValue}.$                                               |

Fonte: O autor, com base em `ControlUnit.v`, `registerBank.v` e `PC_main.v`.

### 3.5 Memórias e temporização

A ROM é uma matriz de 1024 palavras de 32 bits, endereçada pelos dez bits menos significativos do PC e inicializada no fluxo Quartus por `modules/program.mif`. O arquivo ativo pode ser trocado pelos alvos de Make que selecionam `program1.mif` a `program10.mif`. A descrição antiga baseada em arquivo `.txt` não corresponde à versão atual.

A RAM tem profundidade de 64 endereços. Há, contudo, uma inconsistência estrutural importante: `DataMemory.v` declara largura padrão de 64 bits, enquanto ULA, banco, compilador e simulador usam palavras de 32 bits. As conexões truncam ou estendem valores implicitamente. Portanto, este relatório não normaliza a RAM como “64 palavras de 32 bits”; registra a intenção de 32 bits e a declaração RTL atual de 64 bits como uma pendência de correção e validação.

Também não existe hoje uma separação física comprovada entre pilha e dados globais. O back-end reserva conceitualmente 64 palavras para a pilha e inicia os globais no endereço 64, mas o RTL usa somente seis bits de endereço e o simulador aplica máscara de 64 posições. Assim, o endereço 64 volta à posição zero. Os testes atuais podem passar quando os intervalos efetivamente usados não colidem, mas isso não constitui isolamento de regiões. O mapa de memória precisa ser redefinido antes de sustentar processos, pilhas múltiplas ou proteção.

O projeto usa pulsos derivados de `CLOCK_50`: um clock efetivo mais lento e um `fast_clock`. Há lógica acionada por bordas desses dois sinais e não existe arquivo SDC no conjunto atual. Essa arquitetura requer revisão de temporização e CDC antes que o hardware seja usado como base confiável para mecanismos de SO.

## 4 Estado atual do compilador

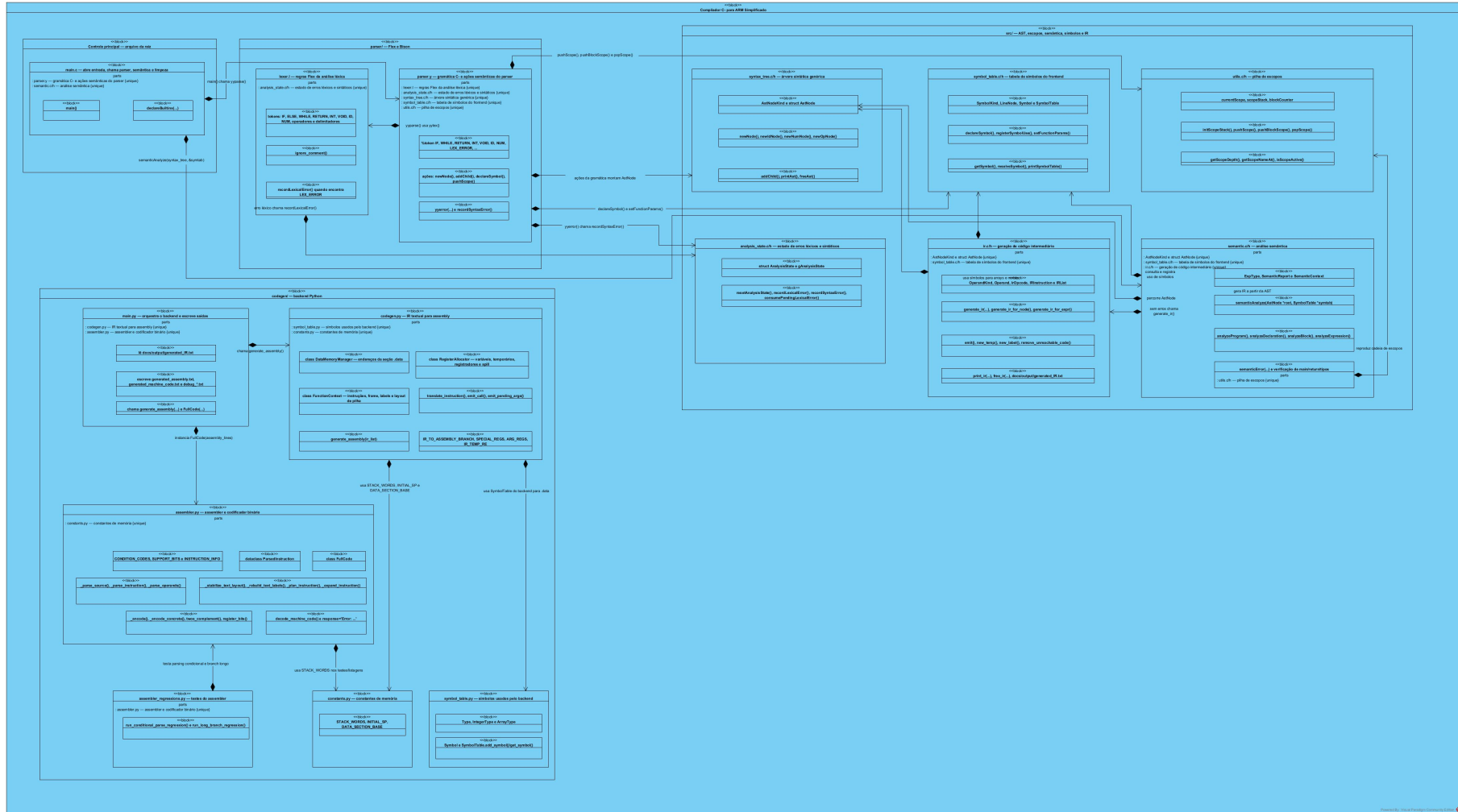
O compilador é uma cadeia de dois programas coordenados por arquivos intermediários. O executável C realiza análise léxica, sintática e semântica, mantém a AST e a tabela de símbolos e só emite IR quando o programa é válido. Em seguida, o back-end Python consome essa IR, constrói sua visão de símbolos e memória, gera assembly e chama o montador. O produto final é uma sequência de palavras de 32 bits que pode alimentar o simulador de máquina e o gerador de MIF.

### 4.1 Arquitetura dos módulos

A [Figura 5](#) complementa a visão sequencial com os arquivos e estruturas concretos da implementação. No front-end, `main.c` coordena o lexer e o parser; as ações da gramática constroem nós de `syntax_tree`, registram declarações e manipulam a pilha de escopos. `semantic.c` percorre a AST com apoio da tabela de símbolos, enquanto `ir.c` emite a representação intermediária. No back-end, `codegen.py` reúne o gerenciador de dados, o alocador de registradores e o contexto de funções; `assembler.py` estabiliza o layout, resolve rótulos e codifica as instruções.

O diagrama também evidencia duas tabelas de símbolos distintas. A estrutura em C registra declarações, tipos, escopos, linhas de uso e assinaturas durante a análise. A estrutura em Python é reconstruída a partir da IR para apoiar endereços, tipos e geração de código. Elas cooperam por meio do contrato textual da IR, mas não compartilham memória durante a execução.

Figura 5 – Hierarquia dos módulos do compilador e suas relações atuais



Fonte: relatório de compiladores, conferido contra os arquivos atuais em `compiler/`. A figura representa dependências e responsabilidades; os artefatos sob `compiler/docs/generated/` são saídas, não fontes editáveis.

## 4.2 Fase de análise

O front-end é escrito em C. Flex reconhece palavras-chave, identificadores, inteiros, operadores, delimitadores e comentários; Bison aplica a gramática C- e constrói a AST. A tabela de símbolos representa funções, variáveis, vetores e escopos aninhados. A análise semântica verifica declarações, tipos, chamadas, vetores, retornos e a presença de `main`. Erros léxicos, sintáticos ou semânticos impedem a geração de IR.

Tabela 5 – Módulos da fase de análise

| Módulo                                                 | Responsabilidade                                                             |
|--------------------------------------------------------|------------------------------------------------------------------------------|
| <code>lexer.l</code> e <code>parser.y</code>           | Tokens, gramática, construção inicial da AST e abertura de escopos.          |
| <code>syntax_tree.c/h</code>                           | Representação, impressão e liberação da AST.                                 |
| <code>symbol_table.c/h</code> e <code>utils.c/h</code> | Símbolos, assinaturas, usos e pilha de escopos.                              |
| <code>semantic.c/h</code>                              | Validação semântica e autorização para gerar IR.                             |
| <code>ir.c/h</code>                                    | IR de três endereços, temporários, rótulos e remoção de código inalcançável. |

Fonte: O autor, com base no código atual.

### 4.2.1 Análise léxica e controle de erros

O scanner em `lexer.l` converte o fluxo de caracteres em tokens para palavras-chave, identificadores, números, operadores e delimitadores. Comentários são consumidos sem alcançar a gramática. Quando encontra um caractere inválido, o lexer registra o erro no estado de análise. Esse estado permite que o parser reconheça que determinada falha nasceu na etapa léxica e evita que um único caractere gere uma sequência enganosa de diagnósticos sintáticos.

### 4.2.2 Análise sintática e AST

A gramática em `parser.y` reconhece declarações globais, funções, parâmetros, blocos, comandos condicionais e iterativos, retornos e expressões. Suas ações semânticas criam nós da AST e registram informações necessárias à resolução posterior. A árvore elimina detalhes puramente gramaticais e preserva a estrutura relevante para tipos, controle de fluxo e geração da IR. O módulo `syntax_tree.c` centraliza a criação, o encadeamento, a impressão diagnóstica e a liberação desses nós.

### 4.2.3 Símbolos, escopos e análise semântica

A pilha de escopos distingue o nível global, funções e blocos aninhados. Cada símbolo registra sua categoria, tipo, escopo de declaração, linhas de uso e, para funções, sua assinatura. A análise semântica percorre a AST para rejeitar redeclarações incompatíveis,

identificadores ausentes, índices ou argumentos inválidos, usos inconsistentes de vetores, retornos incompatíveis e programas sem `main`. Ela também verifica caminhos de retorno em funções que produzem valor. Somente depois dessa etapa a IR é autorizada, preservando a regra de que um programa inválido não segue para o back-end.

## 4.3 IR, back-end e montagem

A IR textual é o contrato entre o front-end em C e o back-end em Python. Ela representa atribuição, endereço, load/store, declaração, aritmética, comparação, rótulos, desvios, argumentos, chamadas e retornos. O operador módulo é reduzido a divisão, multiplicação e subtração porque não existe opcode nativo correspondente.

### 4.3.1 Representação intermediária

O emissor de IR usa temporários e rótulos para tornar explícita a ordem de avaliação. Expressões compostas são decompostas em operações de três endereços; condicionais e laços tornam-se comparações seguidas de branches e labels; acessos a vetores distinguem obtenção de endereço e leitura ou escrita do valor. Declarações globais também aparecem na IR para que o back-end possa reservar a seção de dados. Antes da impressão, uma passagem local remove instruções inalcançáveis posteriores a desvios incondicionais ou retornos até o próximo rótulo.

O back-end calcula frames, aloca registradores, derrama temporários e emite assembly. A convenção reserva `r0` para retorno, `r1-r3` para argumentos rápidos, `r27` para expansões do montador, `r28` como link lógico, `r29` como SP, `r30` como auxiliar de spill e `r31` como FP. O montador resolve rótulos, expande imediatos e branches longos, e codifica as palavras finais.

### 4.3.2 Frames, registradores e chamadas

Cada função possui um contexto que associa parâmetros, locais, vetores e temporários a registradores ou posições relativas ao frame pointer. Os três primeiros argumentos usam `r1-r3`; excedentes são colocados na pilha. `r0` transporta o retorno. Quando os registradores temporários se esgotam, o alocador cria posições de *spill* no frame e emite cargas ou armazenamentos por meio do auxiliar `r30`. O prólogo preserva o frame anterior e o retorno lógico, e o epílogo restaura esse estado antes de transferir o controle ao chamador.

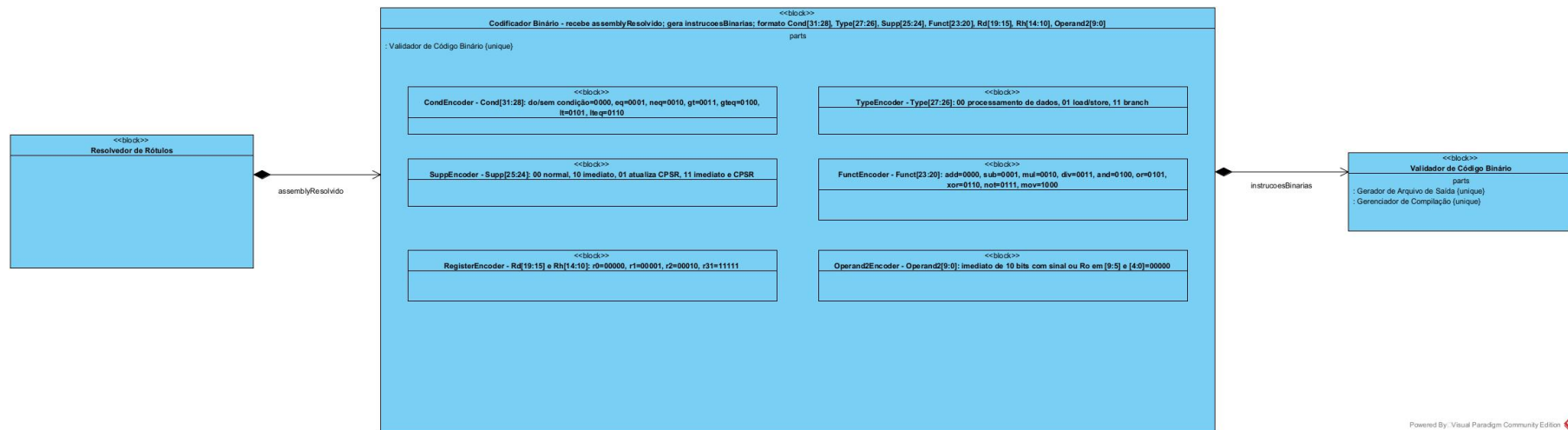
Existe uma distinção essencial: `r28` é um registrador geral usado pela convenção do compilador, enquanto o RTL mantém também um link dedicado. O prólogo copia o retorno para `r28` quando necessário; o epílogo restaura esse valor e emite branch por registrador. Essa ponte deve permanecer documentada para evitar confundir as duas entidades.

### 4.3.3 Montagem e codificação binária

O montador faz mais do que substituir mnemônicos por bits. Primeiro separa `.text` e `.data`, reconhece rótulos e diretivas e calcula os endereços. Depois repete o cálculo de layout quando um imediato ou branch precisa ser expandido em mais de uma instrução. Só então codifica condição, classe, suporte, função e operandos. Esse processo iterativo é necessário porque a expansão de uma instrução altera o endereço das instruções e dos rótulos seguintes.

A [Figura 6](#) apresenta a decomposição funcional da última etapa. Os blocos internos são uma modelagem do comportamento de `assembler.py`, e não classes separadas no código. A figura continua válida para os campos de 32 bits e para os sete sufixos condicionais aceitos. A semântica especial de branch e retorno permanece detalhada na [Tabela 4](#), pois não cabe integralmente nessa visão compacta.

Figura 6 – Decomposição funcional do montador e do codificador binário



Powered By: Visual Paradigm Community Edition

Fonte: relatório de compiladores, validado contra `compiler/codegen/assembler.py`. Os encoders mostrados são responsabilidades conceituais implementadas pelo montador.

#### 4.3.4 Exemplo validado de assembly e código de máquina

A Figura 7 preserva um exemplo visual do relatório de compiladores, mas sua correspondência foi novamente verificada com o montador atual. As doze linhas exercitam imediatos, operações aritméticas, atualização de flags, branch condicional e branch incondicional. Ao executar essas mesmas linhas em `FullCode`, o montador produziu com sucesso as doze palavras mostradas, sem diferença bit a bit.

```

1  movi: r1 = 0
2  movi: r2 = 1
3  movi: r3 = 8
4  mov: r0 = r3
5  add: r2 = r1, r2
6  sub: r1 = r2, r1
7  subi: r3 = r3, 1
8  subis: r16 = r3, 0
9  bineq: -7
10 mov: r0 = r3
11 mov: r31 = r2
12 bi: -1

```

(a) Assembly de entrada.

```

1  00000010100000001000000000000000
2  00000010100000001000000000000001
3  000000101000000011000000000001000
4  00000000100000000000000001100000
5  000000000000000010000010001000000
6  000000000000100001000010000010000
7  00000010000100011000110000000001
8  00000011000110000000110000000000
9  00101110000000000000000111111001
10 00000000100000000000000001100000
11 00000000100011111000000001000000
12 00001110000000000000000111111111

```

(b) Palavras binárias produzidas.

Figura 7 – Correspondência validada entre assembly e código de máquina

Fonte: acervo do relatório de compiladores; resultado reproduzido com `compiler/codegen/assembler.py` em 26 de setembro de 2026.

## 4.4 Integração com ROM e modelo executável

O gerador de MIF converte código binário em imagens de 1024 palavras de 32 bits. O simulador Python reproduz a decodificação, registradores, link, PC, flags Z/N, RAM de 64 posições, entrada e saída. Ele é evidência funcional importante, mas não prova temporização, inferência de memória, CDC, pinagem ou funcionamento físico da FPGA.

## 4.5 Verificação reproduzida nesta consolidação

Em 23 de setembro de 2026 foram executadas três verificações a partir da raiz do repositório. `make -C compiler test_analysis` aprovou os casos válido, léxico, sintático, semântico, identificador não declarado e ausência de `main`; apenas a geração opcional do modelo VPP foi ignorada porque o artefato não estava presente. `python3 compiler/codegen/assembler_regressions.py` aprovou as regressões do montador. Por



fm, `make -C compiler run_selected_10_diagnostics` gerou e executou os dez programas selecionados, com dez resultados esperados em dez casos.

Esses resultados sustentam a coerência funcional entre front-end, back-end, montador e modelo Python para os casos cobertos. Eles não substituem simulação do RTL, compilação Quartus, análise de timing ou teste físico em FPGA, que não foram executados nesta consolidação.

## 5 Comparação dos relatórios e correções

Tabela 6 – Síntese da comparação entre os relatórios e o projeto atual

| Tema                | Processador 2024                          | Compiladores 2025                       | Decisão atual                                                                            |
|---------------------|-------------------------------------------|-----------------------------------------|------------------------------------------------------------------------------------------|
| Fundamentação       | Harvard, RISC e ARM                       | Resume e integra ao C-                  | Mantida como contexto, sem alegar compatibilidade ARM.                                   |
| ISA                 | Evolução histórica e sufixos              | Campos de 32 bits e integração          | Mantida somente na forma confirmada pelo montador e RTL.                                 |
| Datapath            | Diagrama antigo e link externo            | Reutiliza o diagrama                    | Figura rejeitada; descrição reconstruída de <b>integrated.v</b> .                        |
| ROM                 | Inicialização textual antiga              | Menciona arquivo textual em um trecho   | Corrigida para <b>program.mif</b> , 1024 por 32.                                         |
| RAM                 | Descrita genericamente                    | Afirma 64 por 32                        | Registrada divergência: RTL 64 por 64, software 64 por 32.                               |
| Branch              | Semântica anterior parcialmente histórica | Diferencia imediato, registrador e link | Atualizada para PC relativo, destino absoluto em registrador e link dedicado.            |
| Compilador          | Apenas assembler inicial                  | Pipeline C-/IR/Python completo          | Conteúdo exclusivo e complementar do relatório de compiladores.                          |
| Testes              | Waveforms e bancada de 2024               | Exemplos e simulador                    | Resultados antigos preservados como históricos; estado atual exige nova síntese/bancada. |
| Sistema Operacional | Não abordado                              | Não abordado                            | Declarado não implementado.                                                              |

Fonte: O autor. A matriz expandida está em **COMPARACAO.md**.

As 35 imagens disponíveis nas duas pastas do relatório de compiladores foram avaliadas individualmente. Oito foram incorporadas: placa, comparação Harvard/von Neumann, fluxo, hierarquia, codificador, waveform compatível da ULA e o par assembly/código de máquina. Permanecem rejeitadas as figuras antigas de *condition field*, conjunto de instruções e caminho de dados, pois contêm divergências materiais como condição não suportada, PC por quatro, roteamento antigo da RAM ou codificação anterior. Formas de onda sem revisão ou testbench identificável também não foram tratadas como validação atual. A decisão nominal para cada arquivo está registrada em **COMPARACAO.md**.

## 6 Lacunas para o próximo Laboratório

### 6.1 Lacunas do hardware e da documentação

Antes de introduzir mecanismos de SO, a plataforma precisa de uma base mais determinística:

- produzir um diagrama atual do caminho de dados a partir do RTL validado;
- alinhar a largura da RAM entre RTL, compilador, MIF e simulador;
- eliminar o alias entre a pilha de 64 palavras e a seção global iniciada no endereço 64;
- definir e testar o estado após reset para registradores, link, flags, RAM e saída;
- substituir ou justificar a distribuição de clocks derivados e criar restrições SDC;
- confirmar por síntese a inferência de ROM, RAM, multiplicação e divisão;
- criar testbenches autochecking para ISA, branches, memória, entrada e saída;
- executar nova compilação Quartus e nova validação em placa antes de reutilizar resultados de 2024.

### 6.2 Lacunas de interface necessárias a um SO

O repositório não apresenta ainda uma arquitetura de Sistema Operacional. Não foram identificados mecanismos completos de modo usuário/supervisor, exceções, tabela de vetores, instrução de retorno de interrupção, temporizador com protocolo de interrupção, chamadas de sistema, proteção de memória ou troca de contexto. O programa `10_carga_preempcao` é uma carga diagnóstica; seu nome não comprova preempção em hardware ou software.

Consequentemente, o próximo Lab deve começar pela especificação, e não pela documentação retrospectiva de funcionalidades inexistentes. As decisões iniciais precisam definir o modelo de execução, o contexto salvável, o evento de interrupção, o ABI de chamadas de sistema e o mapa de memória. Somente depois dessas decisões o relatório poderá afirmar a existência de processos, escalonador ou serviços de SO.

## 6.3 Critérios mínimos para a próxima revisão

Uma revisão futura deste documento deverá exigir evidência reproduzível para cada nova capacidade: requisito, alteração de ISA/RTL quando aplicável, produtor e consumidor atualizados, teste de software, simulação RTL, síntese e, para afirmações de bancada, execução física documentada.

## 7 Conclusão

Os relatórios anteriores são complementares: o primeiro explica a origem arquitetural e os módulos do processador; o segundo conecta essa máquina a um compilador C- mais amplo. A comparação com o repositório atual, porém, mostra que várias descrições visuais e operacionais ficaram desatualizadas. O principal resultado desta consolidação é separar o que permanece válido do que precisa ser novamente projetado ou comprovado.

A plataforma já possui uma cadeia funcionalmente testável de C- até código de máquina e um RTL capaz de consumir a mesma codificação. Ela ainda não possui um Sistema Operacional. Este relatório, portanto, encerra a documentação do estado inicial e fornece uma lista explícita de riscos e lacunas para que o próximo laboratório avance sobre uma base verificável.

## Referências Bibliográficas

ARM. *ARM7TDMI-S Technical Reference Manual*. [S.l.], 2001. Citado na página 9.

HARRIS, S. L.; HARRIS, D. M. *Digital Design and Computer Architecture*. 1. ed. Burlington: Elsevier, 2016. Citado na página 9.

LOUDEN, K. C.; SILVA, F. S. C. da. *Compiladores: princípios e práticas*. São Paulo: Cengage Learning, 2004. Citado na página 7.

PATTERSON, D. A.; HENNESSY, J. L. *Computer Organization and Design: The Hardware/Software Interface*. [S.l.]: Morgan Kaufmann, 2014. Citado 2 vezes nas páginas 7 e 9.

PATTERSON, D. A.; HENNESSY, J. L. *Organização e Projeto de Computadores: A interface hardware/software*. 5. ed. Rio de Janeiro: GEN LTC, 2017. Citado na página 7.

TERASIC TECHNOLOGIES INC. *DE2-115 User Manual*. [S.l.], 2012. Citado na página 13.